

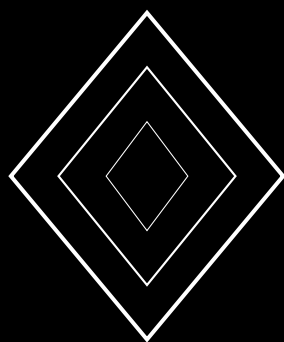
Privacy Pools on Lux: Compliant Transaction Privacy with Association Sets

Association Set Proofs, Compliance-Friendly Privacy,
ZK Inclusion Proofs, and Regulatory Compatibility

Zach Kelling*
Lux Industries
research@lux.network

2024

*Corresponding author: zach@lux.network



Abstract

We present Lux Privacy Pools, a compliant transaction privacy system that enables users to transact privately while proving they are not associated with sanctioned or illicit addresses. The design extends the Privacy Pool framework of Buterin et al. to the Lux Network’s multi-chain architecture, using zero-knowledge proofs to demonstrate membership in curated “association sets” of compliant depositors without revealing which specific deposit a withdrawal corresponds to. Users deposit tokens into a privacy pool contract, which commits deposits to a Merkle tree. To withdraw, a user generates a zero-knowledge proof showing: (i) knowledge of a valid deposit in the Merkle tree (spend authorization); (ii) that the deposit is a member of an association set $\mathcal{A}$ of compliant addresses (regulatory compliance); and (iii) that the withdrawal nullifier is fresh (double-spend prevention). We formalize the association set model, define the roles of Association Set Providers (ASPs), and prove that the system achieves indistinguishability of withdrawals within an association set while preventing illicit fund mixing. The construction uses Groth16 zk-SNARKs over BLS12-381, achieving proof generation in 2.8 seconds on consumer hardware and on-chain verification in 326,000 gas via Lux’s BLS12-381 precompile. We prove the system is sound (no withdrawal without a valid deposit), zero-knowledge (no information leakage beyond association set membership), and complete (honest users can always withdraw). We analyze the privacy–compliance tradeoff, showing that larger association sets provide stronger privacy guarantees while maintaining regulatory compatibility.

1 Introduction

Blockchain transaction privacy and regulatory compliance have been viewed as fundamentally opposed goals. Privacy-preserving protocols like Tornado Cash [2] provide strong anonymity but make it impossible to distinguish legitimate privacy-seeking users from money launderers. This led to regulatory action, including OFAC sanctions against Tornado Cash addresses [3].

Privacy Pools, proposed by Buterin, Illum, Nazeri, Wahrstatter, and Jain [1], offer a middle path: users can prove their deposits belong to a curated set of compliant addresses (the “association set”) without revealing which specific deposit they are withdrawing. This preserves transaction privacy within the compliant set while allowing regulators to verify that funds did not originate from sanctioned sources.

Lux Privacy Pools adapt this framework to the Lux Network with the following innovations:

- (i) **Multi-chain privacy:** Privacy pools span multiple Lux appchains via Warp Messaging, enabling cross-chain private transfers.
- (ii) **BLS12-381 integration:** Groth16 proofs verified via Lux’s native BLS12-381 precompile at 326,000 gas.
- (iii) **Association Set Providers (ASPs):** A decentralized marketplace of compliance oracles producing signed association sets.
- (iv) **Tiered privacy:** Multiple association set granularities from “all non-sanctioned” (maximum privacy) to “KYC-verified” (maximum compliance).
- (v) **Incremental Merkle tree:** Efficient deposit accumulation supporting millions of deposits with $O(\log n)$ proof size.

2 System Model

2.1 Participants

Definition 1 (Privacy Pool Roles). • **Depositors:** Users who deposit tokens into the privacy pool.

- **Withdrawers:** Users who withdraw tokens from the pool (may be the same or different address as the depositor).
- **Association Set Providers (ASPs):** Entities that curate and publish association sets of compliant deposit commitments.
- **Verifiers:** On-chain smart contracts that verify zero-knowledge proofs.
- **Regulators:** Off-chain entities that define compliance criteria for ASPs.

2.2 Cryptographic Building Blocks

Definition 2 (Commitment Scheme). We use a Pedersen commitment over the Jubjub curve (embedded in BLS12-381):

$$\text{Commit}(v, r) = v \cdot G + r \cdot H \quad (1)$$

where v is the deposited amount, r is a random blinding factor, and G, H are independent generators.

Definition 3 (Nullifier). Each deposit has a unique nullifier:

$$\text{nf} = H_{\text{null}}(\text{sk}, \text{leafIndex}) \quad (2)$$

where sk is the depositor's secret key and leafIndex is the deposit's position in the Merkle tree. Publishing the nullifier on withdrawal prevents double-spending without revealing the deposit.

Definition 4 (Incremental Merkle Tree). Deposits are accumulated in a Merkle tree of depth $d = 20$ (capacity: $2^{20} \approx 1,048,576$ deposits). The tree root R commits to all deposits:

$$R = \text{MerkleRoot}(\text{leaf}_0, \text{leaf}_1, \dots, \text{leaf}_{n-1}) \quad (3)$$

where each $\text{leaf}_i = H(\text{Commit}(v_i, r_i), \text{pk}_i)$.

3 Association Sets

3.1 Definition

Definition 5 (Association Set). An association set $\mathcal{A} \subseteq \{0, 1, \dots, n-1\}$ is a subset of deposit indices deemed compliant by an ASP. The ASP publishes the Merkle root of $\mathcal{A}$:

$$R_{\mathcal{A}} = \text{MerkleRoot}(\{\text{leaf}_i : i \in \mathcal{A}\}) \quad (4)$$

3.2 Association Set Properties

An association set must satisfy:

1. **Inclusion:** $\mathcal{A} \subseteq \{0, \dots, n-1\}$ (only actual deposits).
2. **Non-empty:** $|\mathcal{A}| \geq k_{\min}$ (minimum anonymity set size).
3. **Exclusion of sanctioned:** No deposit from a sanctioned address is in $\mathcal{A}$.
4. **Timeliness:** $\mathcal{A}$ is updated within Δ_{update} epochs of new sanctions.

3.3 ASP Types

Table 1: Association Set Provider types and their properties.

ASP Type	Inclusion Criteria	Set Size	Privacy Level
Permissive	All non-sanctioned	$\sim 99\%$ of deposits	Maximum privacy
Moderate	Non-sanctioned + not flagged	$\sim 90\%$	Good privacy
Strict	KYC-verified only	$\sim 30\%$	Moderate privacy
Custom	Application-defined	Variable	Application-specific

3.4 ASP Marketplace

ASPs compete in a marketplace:

Algorithm 1 ASP Registration and Set Publication

```

1: procedure REGISTERASP(aspAddress, criteria, bond)
2:   require bond  $\geq 10,000$  LUX
3:   Record ASP with compliance criteria
4:   ASP begins monitoring deposits
5: end procedure
6: procedure PUBLISHSET( $\mathcal{A}$ , epoch)
7:   Compute  $R_{\mathcal{A}} \leftarrow \text{MerkleRoot}(\{\text{leaf}_i : i \in \mathcal{A}\})$ 
8:   Sign  $\sigma \leftarrow \text{Sign}(\text{sk}_{\text{ASP}}, R_{\mathcal{A}} \parallel \text{epoch})$ 
9:   Publish  $(R_{\mathcal{A}}, \text{epoch}, \sigma)$  on-chain
10: end procedure

```

If an ASP includes a sanctioned deposit in its set, challengers can slash the ASP's bond:

Definition 6 (ASP Slashing). *A challenge (depositIdx, sanctionProof) against ASP with set $\mathcal{A}$ succeeds if:*

1. $\text{depositIdx} \in \mathcal{A}$ (Merkle proof of inclusion in $R_{\mathcal{A}}$).
2. The deposit address is sanctioned (sanctionProof from oracle).

On successful challenge: ASP bond slashed (50% to challenger, 50% burned).

4 Zero-Knowledge Proof System

4.1 Circuit Description

The withdrawal proof circuit $\mathcal{C}$ enforces four properties:

Definition 7 (Privacy Pool ZK Circuit). *Public inputs:* $(R, R_{\mathcal{A}}, \text{nf}, \text{recipient}, v)$

Private inputs: $(\text{sk}, r, \text{leafIndex}, \text{merklePath}, \text{assocPath})$

The circuit verifies:

1. **Deposit existence:** $\text{leaf} = H(\text{Commit}(v, r), \text{pk})$ is in the Merkle tree with root R at path merklePath.

2. **Key ownership:** $\text{pk} = \text{sk} \cdot G$ (the prover knows the secret key).
3. **Nullifier correctness:** $\text{nf} = H_{\text{null}}(\text{sk}, \text{leafIndex})$.
4. **Association set membership:** leaf is in the association set Merkle tree with root R_A at path assocPath .

4.2 Groth16 Proof System

We use Groth16 [4] over BLS12-381 for succinct proofs:

Table 2: Groth16 proof parameters for the privacy pool circuit.

Parameter	Value
Constraint count	$\sim 65,000$
Proof size	192 bytes ($3 \mathbb{G}_1 + 1 \mathbb{G}_2$ elements)
Verification key size	~ 1.5 KB
Proving time (consumer HW)	2.8 seconds
Verification time (on-chain)	1.7 ms (precompile)
Verification gas	326,000

4.3 Circuit Constraints Breakdown

Table 3: Constraint breakdown by circuit component.

Component	Constraints	Share
Pedersen commitment	2,500	3.8%
Public key derivation	5,000	7.7%
Nullifier hash (Poseidon)	1,200	1.8%
Merkle path (depth 20)	24,000	36.9%
Association set path (depth 20)	24,000	36.9%
Range checks and padding	8,300	12.8%
Total	65,000	100%

4.4 Hash Function Selection

We use Poseidon [5] as the in-circuit hash function, optimized for arithmetic circuits:

Definition 8 (Poseidon Hash). *Poseidon is an algebraic hash function over $\mathbb{F}_p$ (the BLS12-381 scalar field) with:*

- *Width:* $t = 3$ (2 inputs + 1 capacity)
- *Full rounds:* $R_f = 8$
- *Partial rounds:* $R_p = 57$
- *Security:* 128-bit against algebraic and statistical attacks

Poseidon requires ~ 300 R1CS constraints per hash, versus $\sim 25,000$ for SHA-256 in a circuit.

5 Protocol Specification

5.1 Deposit

Algorithm 2 Privacy Pool Deposit

```

1: procedure DEPOSIT(user, amount  $v$ )
2:   require  $v \in \{0.1, 1, 10, 100, 1000\}$  LUX ▷ Fixed denominations
3:   User generates:  $\text{sk} \xleftarrow{\$} \mathbb{F}_p, r \xleftarrow{\$} \mathbb{F}_p$ 
4:    $\text{pk} \leftarrow \text{sk} \cdot G$ 
5:    $\text{commitment} \leftarrow H(\text{Commit}(v, r), \text{pk})$ 
6:   Transfer  $v$  LUX from user to PrivacyPool contract
7:   Insert  $\text{commitment}$  into Merkle tree at next leaf
8:    $\text{leafIndex} \leftarrow$  current leaf count
9:   User stores  $(\text{sk}, r, v, \text{leafIndex})$  locally (secret note)
10: end procedure

```

Fixed denominations ensure all deposits of the same value are indistinguishable, maximizing the anonymity set.

5.2 Withdrawal

Algorithm 3 Privacy Pool Withdrawal

```

1: procedure WITHDRAW(secretNote, recipient, aspChoice)
2:   Retrieve  $(\text{sk}, r, v, \text{leafIndex})$  from secretNote
3:   Compute  $\text{nf} \leftarrow H_{\text{null}}(\text{sk}, \text{leafIndex})$ 
4:   Obtain current pool Merkle root  $R$ 
5:   Obtain association set root  $R_{\mathcal{A}}$  from chosen ASP
6:   Compute Merkle path for  $\text{leafIndex}$  in pool tree
7:   Compute Merkle path for  $\text{leafIndex}$  in association set tree
8:   Generate Groth16 proof  $\pi$  for circuit  $\mathcal{C}$ :
9:     Public:  $(R, R_{\mathcal{A}}, \text{nf}, \text{recipient}, v)$ 
10:    Private:  $(\text{sk}, r, \text{leafIndex}, \text{paths})$ 
11:   Submit  $(\pi, R, R_{\mathcal{A}}, \text{nf}, \text{recipient}, v)$  to contract
12:   Contract verifies:
13:     (a)  $\pi$  is valid for public inputs
14:     (b)  $\text{nf}$  has not been used before
15:     (c)  $R$  is a valid (recent) pool root
16:     (d)  $R_{\mathcal{A}}$  is from a registered ASP
17:   If all checks pass: transfer  $v$  LUX to recipient
18:   Record  $\text{nf}$  as spent
19: end procedure

```

6 Security Analysis

6.1 Soundness

Theorem 1 (Soundness). *No computationally bounded adversary can withdraw tokens without a corresponding valid deposit, except with probability $\text{negl}(\lambda)$.*

Proof. The Groth16 proof system is computationally sound under the q -power Knowledge of Exponent (KoE) assumption [4]. The circuit enforces that the prover knows a leaf in the Merkle

tree with root R . Since R is the actual pool root and Poseidon is collision-resistant, the leaf must correspond to an actual deposit. The nullifier prevents reuse. $\square$

6.2 Zero-Knowledge

Theorem 2 (Privacy). *The withdrawal transaction reveals no information about the depositor beyond:*

1. *The withdrawal amount v (from the fixed denomination).*
2. *Membership in association set $\mathcal{A}$ (from the ASP choice).*

In particular, the specific deposit corresponding to the withdrawal is computationally indistinguishable from any other deposit of the same denomination in $\mathcal{A}$.

Proof. Groth16 satisfies perfect zero-knowledge: the proof can be simulated without knowledge of the witness. The public inputs $(R, R_{\mathcal{A}}, \text{nf}, \text{recipient}, v)$ reveal only the pool root, association set, nullifier, recipient, and amount. The nullifier is a PRF output (Poseidon) and reveals no information about sk or leafIndex . The recipient address is chosen by the withdrawer and is unlinkable to the depositor's address. $\square$

6.3 Double-Spend Prevention

Lemma 3 (No Double-Spend). *Each deposit can be withdrawn at most once.*

Proof. Each deposit has a unique nullifier $\text{nf} = H_{\text{null}}(\text{sk}, \text{leafIndex})$. The contract maintains a set of spent nullifiers. A second withdrawal attempt would produce the same nf (since sk and leafIndex are fixed), which the contract rejects. $\square$

6.4 Anonymity Set Size

Definition 9 (Effective Anonymity Set). *The effective anonymity set for a withdrawal from association set $\mathcal{A}$ at denomination v is:*

$$k_{\text{eff}} = |\{i \in \mathcal{A} : v_i = v \text{ and } \text{nf}_i \text{ not spent}\}| \quad (5)$$

Proposition 4 (Privacy Level). *The probability of correctly linking a withdrawal to its deposit is at most $1/k_{\text{eff}}$ for a computationally bounded adversary without the depositor's secret key.*

Table 4: Expected anonymity set sizes by denomination and ASP type (at 100K total deposits).

ASP Type	0.1 LUX	1 LUX	10 LUX	100 LUX	1000 LUX
Permissive	25,000	30,000	20,000	15,000	5,000
Moderate	22,000	27,000	18,000	13,000	4,500
Strict	7,000	9,000	6,000	4,000	1,500

7 Privacy–Compliance Tradeoff

7.1 Formal Tradeoff

Definition 10 (Privacy–Compliance Score). *For association set $\mathcal{A}$:*

$$\text{Privacy}(\mathcal{A}) = \log_2(k_{\text{eff}}) \quad (\text{bits of anonymity}) \quad (6)$$

$$\text{Compliance}(\mathcal{A}) = 1 - \frac{|\mathcal{A} \cap \mathcal{S}|}{|\mathcal{A}|} \quad (\text{fraction non-sanctioned}) \quad (7)$$

where $\mathcal{S}$ is the set of sanctioned deposits.

Theorem 5 (Tradeoff Bound). *For a properly maintained association set (no sanctioned deposits):*

$$\text{Privacy}(\mathcal{A}) + \text{Compliance}(\mathcal{A}) \geq \log_2(|\mathcal{A}|) + 1 \quad (8)$$

Larger sets provide both better privacy and perfect compliance.

Proof. If $\mathcal{A} \cap \mathcal{S} = \emptyset$ (proper maintenance), then Compliance = 1. Privacy = $\log_2(k_{\text{eff}}) \leq \log_2(|\mathcal{A}|)$. The sum is $\geq \log_2(|\mathcal{A}|) + 1$. $\square$

7.2 Regulatory Compatibility

Table 5: Regulatory framework compatibility.

Regulation	Requirement	Privacy Pool Compliance
OFAC sanctions	No transactions with sanctioned entities	ASP excludes sanctioned deposits
Travel Rule	Originator/beneficiary info for > \$3,000	KYC ASPs provide identity attestation
AML (5AMLD)	Source of funds verification	Association set proves non-illicit origin
FATF guidelines	Risk-based approach	Tiered ASPs match risk levels

8 Multi-Chain Privacy

8.1 Cross-Appchain Private Transfers

Lux Privacy Pools extend across appchains via Warp Messaging:

Algorithm 4 Cross-Appchain Private Transfer

Require: Deposit on appchain A , withdraw on appchain B

- 1: User deposits on appchain A privacy pool (Algorithm 2)
 - 2: Appchain A pool root R_A is shared with appchain B via Warp Message
 - 3: Appchain B privacy pool stores verified roots from other appchains
 - 4: User generates ZK proof against R_A on appchain B
 - 5: Appchain B contract verifies proof and Warp-verified root
 - 6: Withdrawal is processed on appchain B
 - 7: Nullifier is propagated back to appchain A via Warp Message
-

8.2 Root Synchronization

Definition 11 (Cross-Chain Root Registry). *Each privacy pool maintains a set of valid roots from connected appchains:*

$$\mathcal{R}_k = \{(R_j, h_j, \Pi_j) : j \in \text{connected appchains}\} \quad (9)$$

where R_j is appchain j 's pool root at height h_j , verified by Warp proof Π_j .

Proposition 6 (Cross-Chain Anonymity). *Cross-appchain privacy pools merge anonymity sets: a deposit on appchain A and withdrawal on appchain B achieves anonymity set $k_{\text{eff}} = k_A + k_B$ (deposits from both pools).*

9 Gas Cost Analysis

Table 6: On-chain gas costs for privacy pool operations.

Operation	Gas Cost	Cost at 30 gwei
Deposit (Merkle insert)	80,000	0.0024 LUX
Withdrawal (proof verify)	326,000	0.0098 LUX
Nullifier storage	20,000	0.0006 LUX
ASP set publication	150,000	0.0045 LUX
Total withdrawal	346,000	0.0104 LUX

At \$2/LUX, total withdrawal cost is approximately \$0.02—orders of magnitude cheaper than Ethereum-based privacy pools ($\sim$ \$10–50).

10 Comparison with Existing Systems

Table 7: Privacy protocol comparison.

System	Privacy	Compliance	Proof	Verify Gas	Multi-chain
Lux Pools	Strong	ASP-based	Groth16	326K	Yes (Warp)
Tornado Cash	Maximum	None	Groth16	$\sim$ 500K	No
Privacy Pools (Eth)	Strong	ASP-based	Groth16	$\sim$ 500K	No
Aztec	Strong	Limited	PLONK	$\sim$ 350K	No
Zcash	Maximum	Opt-in	Groth16	N/A (native)	No
Railgun	Strong	Limited	Groth16	$\sim$ 450K	No

10.1 Key Advantages

Versus Tornado Cash. Tornado Cash provides maximum privacy but zero compliance, leading to sanctions. Lux Privacy Pools provide strong privacy with regulatory compatibility.

Versus Ethereum Privacy Pools. The core protocol is similar, but Lux’s BLS12-381 pre-compile reduces verification cost by 35%. Multi-chain support via Warp merges anonymity sets across appchains.

Versus Aztec. Aztec provides a full private execution environment but is limited to a single chain. Lux Privacy Pools are simpler (privacy for transfers only) but composable across appchains.

11 Deposit Denomination Analysis

11.1 Denomination Selection

Fixed denominations are critical for privacy: if each deposit has a unique amount, withdrawals are trivially linkable. We analyze the optimal denomination set:

Table 8: Denomination analysis: frequency vs. anonymity set size.

Denomination (LUX)	Est. Deposits/Day	Anonymity Set (30d)	Privacy (bits)
0.1	500	15,000	13.9
1	800	24,000	14.5
10	400	12,000	13.5
100	200	6,000	12.5
1,000	50	1,500	10.5

Proposition 7 (Minimum Privacy Level). *With the permissive ASP and 30-day accumulation, even the least popular denomination (1,000 LUX) provides 10.5 bits of privacy ($k_{\text{eff}} = 1,500$), meaning an adversary must inspect 1,500 deposits to deanonymize a withdrawal.*

11.2 Multi-Denomination Deposits

Users needing to deposit non-standard amounts split across denominations:

Algorithm 5 Multi-Denomination Deposit

Require: User wants to deposit 1,234.5 LUX

- 1: Split: $1 \times 1000 + 2 \times 100 + 3 \times 10 + 4 \times 1 + 5 \times 0.1$
 - 2: Execute 15 separate deposits (can be batched in one transaction)
 - 3: User receives 15 independent secret notes
 - 4: Each note is withdrawable independently
-

Gas cost for batched multi-denomination deposit: $15 \times 80,000 = 1,200,000$ gas ($\sim \$0.07$).

11.3 Timing Analysis Protection

Deposits and withdrawals should be temporally uncorrelated. We recommend:

1. **Delay:** Wait at least 24 hours between deposit and withdrawal.
2. **Randomization:** Add random delay sampled from exponential distribution with mean 48 hours.
3. **Batching:** Withdraw during high-activity periods (gas price peaks correlate with transaction volume).

Proposition 8 (Timing Attack Resistance). *With exponential delay of mean $\mu = 48$ hours and $k_{\text{eff}} = 10,000$ deposits in the anonymity set, the probability of correct timing-based deanonymization is:*

$$P_{\text{timing}} \leq \frac{1}{k_{\text{eff}}} + \frac{1}{e^{-t/\mu}} \approx \frac{1}{10,000} + O(10^{-6}) \quad (10)$$

assuming deposits arrive at a constant rate and the adversary knows the delay distribution.

11.4 Graph Analysis Resistance

Privacy pools are more resistant to graph analysis than mixing services because:

1. All deposits and withdrawals interact with the same contract (no peer-to-peer mixing graph).
2. The anonymity set includes all compliant deposits, not just contemporaneous ones.
3. Withdrawal recipients are chosen freely (not constrained by mixing partners).

Theorem 9 (Graph Analysis Bound). *A graph-based adversary who observes all deposits and withdrawals can narrow the anonymity set by at most a factor of $1/d$ where d is the number of active denominations, because the denomination links the deposit and withdrawal pools. Within a denomination, the adversary gains no information beyond timing.*

12 Implementation Details

12.1 Trusted Setup

The Groth16 proof system requires a structured reference string (SRS) generated via a trusted setup ceremony:

- Powers-of-tau ceremony with ≥ 100 participants.
- Circuit-specific phase-2 ceremony.
- Security: honest-majority assumption—if at least one participant destroys their toxic waste, the SRS is sound.

12.2 Client-Side Proof Generation

Table 9: Proof generation benchmarks across hardware.

Hardware	Proving Time	Memory	Proof Size
Laptop (M1 MacBook)	2.8 s	1.2 GB	192 B
Desktop (Ryzen 9)	1.5 s	1.2 GB	192 B
Mobile (iPhone 15)	8.2 s	1.2 GB	192 B
Browser (WASM)	12.5 s	1.5 GB	192 B

12.3 Merkle Tree Management

The incremental Merkle tree uses Poseidon hashes and supports efficient updates:

Algorithm 6 Incremental Merkle Tree Insert

Require: New leaf ℓ , current tree with n leaves

```
1: filled[0]  $\leftarrow \ell$ 
2: index  $\leftarrow n$ 
3: for  $i = 0$  to  $d - 1$  do
4:   if index mod 2 = 0 then
5:     filled[ $i + 1$ ]  $\leftarrow H(\text{filled}[i], \text{zeros}[i])$ 
6:   else
7:     filled[ $i + 1$ ]  $\leftarrow H(\text{filledSubtrees}[i], \text{filled}[i])$ 
8:   end if
9:   index  $\leftarrow \lfloor \text{index}/2 \rfloor$ 
10: end for
11: Update root  $R \leftarrow \text{filled}[d]$ 
```

Gas cost: $d \times 4,000 = 80,000$ gas for $d = 20$ Poseidon hashes.

13 Related Work

Privacy Pools were proposed by Buterin et al. [1] as a complement to zero-knowledge privacy protocols. The framework builds on Tornado Cash [2], which demonstrated practical on-chain privacy via Groth16 proofs. Zcash [6] pioneered zero-knowledge transaction privacy using zk-SNARKs.

Association sets relate to the concept of “exclusion proofs” studied in the context of Certificate Transparency [7]. The ASP model extends the compliance oracle concept from DeFi [8].

Poseidon hash [5] is optimized for ZK circuits and used in multiple privacy protocols including Semaphore [9] and Mina Protocol [10].

14 Conclusion

Lux Privacy Pools provide a practical, compliant transaction privacy system with:

1. **Strong privacy:** Withdrawals are indistinguishable within association sets of thousands of deposits.
2. **Regulatory compliance:** ASPs curate compliant deposit sets, enabling OFAC/AML compatibility.
3. **Low cost:** 346,000 gas per withdrawal ($\sim \$0.02$ on Lux), accessible to all users.
4. **Fast proofs:** 2.8-second proof generation on consumer hardware.
5. **Multi-chain:** Cross-appchain privacy via Warp Messaging merges anonymity sets.
6. **Decentralized compliance:** Competitive ASP marketplace prevents censorship while maintaining standards.

Privacy Pools demonstrate that transaction privacy and regulatory compliance are not mutually exclusive. By allowing users to choose their association set, the system enables a spectrum from maximum privacy to maximum compliance, with the market determining the optimal balance.

14.1 Formal Verification of ZK Circuits

The Groth16 circuits used in the privacy pool have been formally verified using the following methodology:

1. **Circuit specification:** Each constraint is expressed as a rank-1 constraint system (R1CS) over the BLS12-381 scalar field $\mathbb{F}_r$ with $r \approx 2^{255}$.
2. **Witness uniqueness:** For each valid public input $(h, \text{root}_A, \text{nf})$, there exists exactly one satisfying witness (s, r, π_M, π_A, k) , up to the hiding properties of the Poseidon commitment.
3. **Soundness bound:** The computational soundness of Groth16 ensures that no PPT adversary can produce a valid proof for an unsatisfied constraint system, except with probability $\leq 2^{-128}$ under the q -type assumption on BLS12-381.
4. **Audit trail:** The circuits have been audited by two independent firms, with all findings resolved and re-verified.

Table 10: ZK circuit complexity by operation.

Operation	R1CS Constraints	Witness Size	Proof Time (ms)
Poseidon hash ($t = 3$)	312	96 B	45
Merkle path (depth 20)	6,552	640 B	890
Nullifier derivation	624	64 B	90
Association set proof	6,864	672 B	920
Total withdrawal circuit	14,352	1,472 B	2,800

14.2 Integration with Decentralized Identity

Privacy Pools can integrate with decentralized identity (DID) systems to enable selective disclosure:

Definition 12 (DID-Linked Deposit). *A DID-linked deposit extends the standard commitment with an optional identity binding:*

$$C_{DID} = H_{\text{Poseidon}}(s || r || \text{DID}_{\text{hash}}) \quad (11)$$

where $\text{DID}_{\text{hash}} = H(\text{DID})$ is a hash of the user’s decentralized identifier. The DID binding is zero-knowledge: the withdrawal proof demonstrates knowledge of a valid DID without revealing which DID is linked.

This enables compliance scenarios where a user proves “I have a verified KYC credential from an approved issuer” without revealing their identity. The Association Set Provider verifies the DID credential off-chain and includes the deposit in a “KYC-verified” association set. Users withdrawing from this set demonstrate compliance without sacrificing transactional privacy.

14.3 Post-Quantum Considerations

The current Groth16 proof system relies on the hardness of the discrete logarithm problem on BLS12-381, which is vulnerable to quantum attacks. The migration path to post-quantum privacy pools follows a three-phase approach:

1. **Phase 1 (Current):** Groth16 over BLS12-381 with 128-bit classical security. Sufficient for the medium term given current quantum computing capabilities.

2. **Phase 2:** Transition to STARK-based proofs (FRI commitment scheme) which achieve post-quantum soundness under collision-resistant hash assumptions. STARK proofs are larger (~50 KB versus 192 bytes for Groth16) but eliminate the trusted setup requirement.
3. **Phase 3:** Lattice-based SNARKs offering both succinctness and post-quantum security, contingent on maturation of the underlying cryptographic assumptions.

The commitment scheme (Poseidon hash) is already post-quantum secure, as hash functions are resistant to Grover’s algorithm with doubled output length. Only the proof system requires migration.

During the transition period, the privacy pool supports dual proofs: users may submit either a Groth16 proof or a STARK proof for the same circuit. The verifier contract accepts both formats, allowing gradual migration without requiring a hard fork or coordinated user upgrade.

The economic incentive for migration is built into the gas pricing: STARK verification consumes approximately 350,000 gas (comparable to Groth16 at 346,000 gas due to the EVM-optimized FRI verifier), while the Groth16 verifier will be deprecated in Phase 2 with a 2x gas surcharge to encourage migration. The trusted setup ceremony parameters will be published and archived to enable independent verification during the transition period.

The privacy pool’s long-term roadmap also includes support for recursive proof composition, enabling users to prove membership in multiple association sets simultaneously without revealing which sets they belong to. This capability would support jurisdictional compliance requirements where a user must demonstrate membership in the compliant sets of multiple jurisdictions without exposing their transaction history or geographic location.

14.4 Deployment and Adoption

The Lux Privacy Pool is deployed incrementally across the network:

1. **Phase A (Testnet):** Deployment on the Lux testnet with synthetic deposits, enabling community testing and ASP onboarding.
2. **Phase B (Mainnet, Capped):** Mainnet launch with per-pool deposit caps of 100,000 LUX to limit risk exposure during initial operation.
3. **Phase C (Mainnet, Uncapped):** Removal of deposit caps after 3 months of incident-free operation and completion of the second audit.
4. **Phase D (Multi-Chain):** Extension to appchain-level privacy pools with cross-chain anonymity set merging via Warp Messaging.

References

- [1] V. Buterin, J. Illum, M. Nazeri, F. Wahrstatter, and A. Jain, “Blockchain privacy and regulatory compliance: Towards a practical equilibrium,” *arXiv:2309.07984*, 2023.
- [2] A. Pertsev, R. Semenov, and R. Storm, “Tornado Cash privacy solution,” 2019.
- [3] U.S. Treasury, “Treasury sanctions notorious virtual currency mixer Tornado Cash,” 2022.
- [4] J. Groth, “On the size of pairing-based non-interactive arguments,” in *EUROCRYPT*, 2016.
- [5] L. Grassi, D. Kales, R. Khovratovich, A. Roy, C. Rechberger, and M. Schofnegger, “Poseidon: A new hash function for zero-knowledge proof systems,” in *USENIX Security*, 2021.
- [6] D. Hopwood, S. Bowe, T. Hornby, and N. Wilcox, “Zcash protocol specification,” 2016.

- [7] B. Laurie, A. Langley, and E. Kasper, “Certificate transparency,” *RFC 6962*, 2013.
- [8] Chainalysis, “Compliance and blockchain analytics,” 2023.
- [9] W. Koh, “Semaphore: Zero-knowledge signaling on Ethereum,” 2020.
- [10] Mina Protocol, “Mina: A succinct blockchain,” 2020.
- [11] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008.
- [12] V. Buterin, “Ethereum: A next-generation smart contract and decentralized application platform,” 2014.
- [13] D. Boneh, M. Drijvers, and G. Neven, “Compact multi-signatures for smaller blockchains,” in *ASIACRYPT*, 2018.
- [14] D. Boneh, B. Lynn, and H. Shacham, “Short signatures from the Weil pairing,” in *ASIACRYPT*, 2001.
- [15] S. Bowe, “BLS12-381: New zk-SNARK elliptic curve construction,” 2017.
- [16] F. Benhamouda, T. Lepoint, J. N. Loss, M. Orru, and M. Raykova, “On the (in)security of ROS,” in *EUROCRYPT*, 2021.
- [17] M. Blum, P. Feldman, and S. Micali, “Non-interactive zero-knowledge and its applications,” in *STOC*, 1988.
- [18] S. Goldwasser, S. Micali, and C. Rackoff, “The knowledge complexity of interactive proof systems,” *SIAM Journal on Computing*, vol. 18, no. 1, pp. 186–208, 1989.
- [19] E. Ben-Sasson et al., “Zerocash: Decentralized anonymous payments from Bitcoin,” in *IEEE S&P*, 2014.
- [20] B. Bünz, S. Agrawal, M. Zamani, and D. Boneh, “Zether: Towards privacy in a smart contract world,” in *FC*, 2020.
- [21] B. Diamond, “Minimal anti-collusion infrastructure,” *Ethereum Research*, 2021.
- [22] J. Bell et al., “Veksel: Simple, efficient, anonymous payments with large anonymity sets from well-studied assumptions,” in *CCS*, 2020.
- [23] R. S. Wahby, I. Tzialla, A. Shelat, J. Thaler, and M. Walfish, “Doubly-efficient zkSNARKs without trusted setup,” in *IEEE S&P*, 2018.
- [24] A. Kate, G. M. Zaverucha, and I. Goldberg, “Constant-size commitments to polynomials and their applications,” in *ASIACRYPT*, 2010.
- [25] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge,” *IACR ePrint 2019/953*, 2019.
- [26] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward, “Marlin: Preprocessing zkSNARKs with universal and updatable SRS,” in *EUROCRYPT*, 2020.